

Less Wire, Less Throughput: Re-evaluating a Boundary-Folded Torus Network-on-Chip

Rahul Ray
B.Tech. CSE (2023), Assam University
Silchar, India
rahulrayconnect@gmail.com

Abstract—A 2D torus gives every router in an on-chip network four neighbours, but it pays for that regularity with wraparound links that run the full width of the die. In earlier undergraduate work we proposed a “modified torus” that keeps the corner wraparounds and folds the remaining boundary links back onto the same edge, which shortens them. That evaluation looked only at hop counts on a single 5×5 grid. Here we re-examine the design properly. We give a rule that extends it to any $n \times n$ grid, measure physical wire length and channel load, and simulate it cycle by cycle against a mesh and a torus. At 8×8 the modified torus uses 21% less wire than a torus and has about the same average hop count, but its saturation throughput under uniform traffic is 45% lower, close to that of a mesh. It does better than the torus under tornado traffic (+50%) and much worse under bit-complement. We also show that the shortest-path BFS routing used originally deadlocks at 20% load, and we replace it with adaptive routing over an up*/down* escape channel. Finally, we treat the placement of boundary links as a search problem. Simulated annealing, REINFORCE and random search, given the same evaluation budget, find no layout that beats the hand design at its own wire budget. They do trace out a steep wire/throughput frontier between it and the torus. Our simulator matches BookSim to within a few percent wherever each traffic class has at least two virtual channels.

Index Terms—network-on-chip, torus, topology, wire length, deadlock, adaptive routing, simulated annealing, reinforcement learning

I. Introduction

On-chip networks replaced shared buses once chips had too many cores for a single bus to serve [2], [3]. The 2D mesh is the default topology because it maps directly onto a tiled floorplan and every link is one tile long. Its weakness is diameter: a packet may need $2(n-1)$ hops on an $n \times n$ mesh. The torus halves the diameter by adding a wraparound link to every row and column. On a flat floorplan, though, each wraparound is $n-1$ tiles long. Long links need repeaters or extra pipeline stages, and they take up routing tracks [4].

Our undergraduate thesis [1] proposed a compromise. Keep the torus wraparounds at the four corners, and link most other boundary routers to another router on the same edge, about half an edge away. Every router still has four ports, and most of the extra links are roughly half as long. The thesis reported that this “modified torus” beats both the mesh and the torus. However, it measured only

BFS hop counts on a hand-written 5×5 adjacency list. It had no notion of wire length, contention or deadlock. Its “throughput” was the reciprocal of hop count.

This paper is the evaluation that the original design should have had. The contributions are:

- A construction rule that extends the topology to any $n \geq 5$. It reproduces the original 5×5 list exactly and matches the thesis figures up to 10×10 (Section III).
- A comparison of mesh, torus and modified torus on wire length, hop count and channel-load throughput for $n = 5, \dots, 10$, followed by cycle-level simulation under four traffic patterns (Sections V-A and V-D).
- A demonstration that the original routing deadlocks, both from its channel dependency graph and in simulation, and a deadlock-free replacement that works on any topology (Section V-B).
- A validation of our simulator against BookSim, including one modelling difference that we trace to its cause (Section V-C).
- A formulation of boundary-link placement as a matching problem, with a comparison of random search, simulated annealing and policy-gradient reinforcement learning on it (Section V-E).

The headline result is a trade-off, not a win. The modified torus saves wire and keeps the torus’s low zero-load latency, but it gives up roughly half of the torus’s saturation throughput. Within that trade-off the hand design holds up well. None of our search methods found a better layout with the same amount of wire.

The simulator, experiments and all raw results are available as open-source code.¹

II. Background

Topologies. A k -ary 2-cube (torus) has diameter k for even k and, under uniform traffic with load evenly balanced, saturates at an injection rate of $8/k$ packets per node per cycle [4]. The mesh saturates at about $4/k$. Folded tori interleave routers so that every link has the same length, at the cost of doubling the length of the short links [4]. Express virtual channels [5] take a different approach: packets bypass the router pipelines of intermediate nodes on a mesh, which cuts the per-hop cost

of long trips. The flattened butterfly [6] instead adds long links with high-radix routers. The modified torus stays at radix 4 and only changes where the boundary links go.

Deadlock. With finite buffers, packets can wait on each other in a cycle. Dally and Seitz showed that deterministic routing is deadlock-free if and only if its channel dependency graph (CDG) is acyclic [7]. Shortest-path routing on a ring or torus produces cyclic CDGs. Duato’s theory allows fully adaptive routing as long as an acyclic escape sub-network is always reachable [8]. Up*/down* routing, introduced for the Autonet LAN [9], provides such an escape network on any connected graph.

Search. We compare simulated annealing [10] with REINFORCE [11]. Reinforcement learning has been used for NoC routing since Q-routing [12]. Here we use it for topology layout.

III. The Topology and Its Design Space

Number the routers of an $n \times n$ grid by (row, column). On a mesh, each corner router has two unused ports and every other boundary router has one, giving $4n$ spare ports in total. Any perfect matching of these ports adds $2n$ links and leaves every router with radix 4. The torus is the matching that pairs each port with the opposite edge.

Construction. The modified torus pairs the ports as follows:

- 1) Each corner keeps both torus wraparounds.
- 2) If n is odd, the middle row and middle column keep their wraparounds.
- 3) On each edge, the remaining positions strictly between the corners are listed in order and split into two halves of equal size. Position i of the first half is linked to position i of the second half.

Each such link spans $\lfloor (n-1)/2 \rfloor$ positions along its edge, so it is about half as long as a wraparound. For $n = 5$ the rule gives exactly the 50 links in the original hand-written code. For $n = 6$ to 10 it matches the figures in the thesis. Fig. 1 shows the 8×8 case. For $n = 4$ the folded links would duplicate mesh links, so the topology is defined only for $n \geq 5$.

IV. Methodology

A. Analytical metrics

Wire length is the Manhattan distance between the two routers of a link, measured in tile pitches, with no folding. For throughput we compute channel loads with each flow split evenly over all of its minimal paths. Let $d(s, t)$ be the hop distance, σ_{st} the number of shortest $s \rightarrow t$ paths, and T_{st} the fraction of node s ’s traffic sent to t . The load on directed channel $c = (u \rightarrow v)$ per unit injection rate is

$$\gamma_c = \sum_{s,t} T_{st} \frac{\sigma_{su} \sigma_{vt}}{\sigma_{st}} \mathbf{1}[d(s, u) + 1 + d(v, t) = d(s, t)], \quad (1)$$

and the throughput estimate is

$$\hat{\Theta} = \left(\max_c \left\{ \max_s \gamma_c, \max_t \sum_s T_{st} \right\} \right)^{-1}, \quad (2)$$

where the second term is the ejection limit of one packet per router per cycle. This is an estimate rather than a bound. Adaptive routing can balance load better than an even split, and real routers lose some bandwidth to contention. For a torus the formula reproduces the textbook $8/k$ result [4], up to a factor of $(N-1)/N$ because we exclude self-traffic, and our test suite checks this.

B. Routing and deadlock

We implement three routing functions. BFS follows one fixed shortest path, with ties broken by neighbour index; this is the original algorithm. Up*/down* builds a BFS tree from the most central router and only allows paths that go zero or more hops up the tree and then zero or more hops down. Adaptive lets packets on virtual channels (VCs) $1..V-1$ take any minimal next hop, choosing the one whose downstream buffer has the most free space. A packet that finds every minimal option full drops into VC 0, where it follows up*/down*. Because buffers hold whole packets, it may move back to an adaptive VC at any later hop. We build the CDG of each routing function and check it for cycles with a standard graph algorithm. For adaptive routing we check the CDG of the escape network, which is what Duato’s condition requires.

C. Cycle-level simulator

We wrote a cycle-level simulator in Python (about 400 lines), modelled on the methodology of [4]. Tools such as BookSim [13] or Garnet [14] would also work. We wanted irregular, searched topologies and the escape scheme above, and a small simulator we could read end to end was easier to verify. We check it against BookSim in Section V-C. Each router is input-queued, with $V = 3$ VCs per port and 4-packet buffers per VC, and uses credit-based flow control. Each cycle, each output port and each input port forwards at most one packet, and the allocator rotates its starting input for fairness. A hop costs one router cycle plus the link delay. Links take one cycle in the default model. In a second “wire” model, a link of length L takes $\lceil L/2 \rceil$ cycles, so long wraparounds pay for their length. Packets are single flits with virtual cut-through.

Traffic is open-loop Bernoulli injection into unbounded source queues. Each run warms up for 1000 cycles, tags packets created during the next 3000, and runs until all tagged packets have been delivered. A point counts as saturated if tagged packets are still in flight 6000 cycles after the window, or if the network delivered less than 95% of the offered load. We report saturation throughput as the highest offered load that is not saturated and whose latency stays below three times the zero-load latency. A run counts as deadlocked if packets are in the network but none moves for 1000 cycles.

D. Topology search

The design space is the set of valid matchings of the $4n$ spare boundary ports: no self-loops, no duplicates of

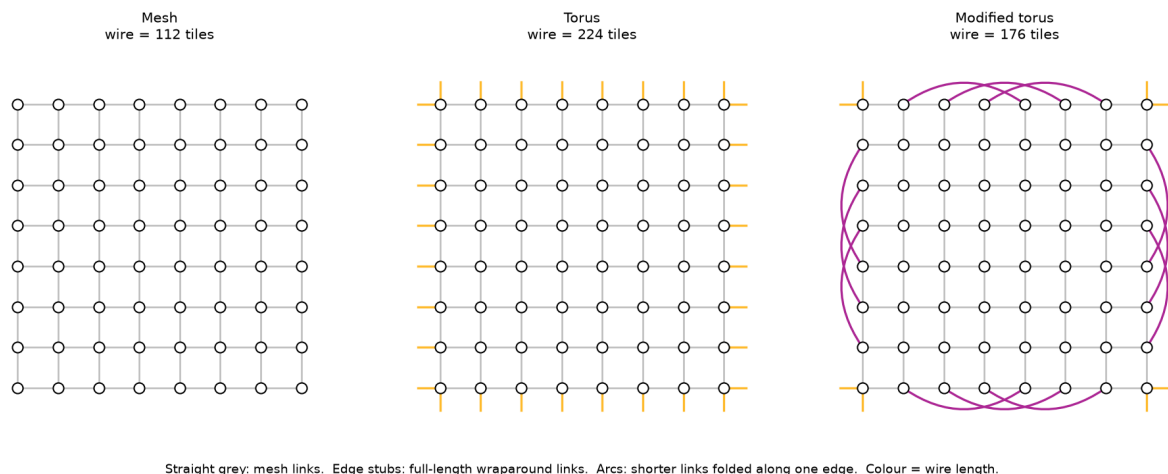


Fig. 1. The three 8×8 networks. Short stubs at the edges stand for full-length wraparound links. Arcs are the shorter folded links. Colour encodes wire length.

mesh links, and no duplicate links. We maximise

$$J(G) = \frac{1}{|P|} \sum_{p \in P} \frac{\hat{\Theta}_p(G)}{\hat{\Theta}_p(\text{torus})} - 0.25 \frac{\bar{h}(G)}{\bar{h}(\text{torus})} - 5 \frac{\max(0, W(G) - B)}{B}, \quad (3)$$

where P is the set {uniform, transpose, bit-complement, tornado}, $\bar{h}$ is the mean hop count, W is total wire length and B is the wire budget. The penalty term steers the search back to feasible designs rather than forbidding infeasible ones outright.

Random search samples matchings uniformly by shuffling the ports and rejecting invalid pairings. Simulated annealing starts from a random or a given matching. Its move picks two links $(a, b), (c, d)$ and re-pairs them as $(a, c), (b, d)$ or $(a, d), (b, c)$, which keeps every router at radix 4. The temperature cools geometrically from 0.05 to 0.001. REINFORCE builds a matching one port at a time. At each step it takes the lowest-numbered unmatched port i and samples a partner j from a softmax over learned logits θ_{ij} , masked to valid partners and mixed with 2% uniform exploration. It trains with a batch-mean baseline and advantages normalised by the batch standard deviation (batch 64, learning rate 8). An episode that reaches a dead end gets a fixed low reward. All three methods get 2500 distinct objective evaluations, and repeated designs are served from a cache for free.

V. Results

A. Wire, hops and channel load

Table I and Fig. 2 summarise the analytical comparison. The original claim about hop count holds, but only just. The modified torus is within 6% of the torus at every size from 5 to 10. It is slightly better at most sizes, the largest gain being 5.9% at 9×9 , and 1.7% worse at 8×8 . The

TABLE I
Analytical metrics. $\hat{\Theta}$ is the uniform-traffic throughput estimate in packets/node/cycle (capped at 1 by ejection).

n	Topology	Diam.	Hops	Wire	$\hat{\Theta}$
8	Mesh	14	5.33	112	0.37
	Torus	8	4.06	224	0.98
	Modified torus	8	4.13	176	0.43
5	Torus	4	2.50	80	1.00
	Modified torus	4	2.49	72	0.95
9	Torus	8	4.50	288	0.89
	Modified torus	8	4.24	240	0.28
10	Torus	10	5.05	360	0.79
	Modified torus	10	4.96	280	0.26

wire saving is real and grows with n : 10% at 5×5 , 21% at 8×8 and 22% at 10×10 .

Throughput is where the two designs part ways. In a torus every channel carries the same load under uniform traffic. In the modified torus, traffic that would have crossed the die on a wraparound is pushed onto the short folded links and the edge routers around them, and those become the bottleneck. At 5×5 the effect is mild (0.95 against 1.00), but by 9×9 the estimate is below that of a plain mesh (0.28 against 0.33). The original evaluation, which measured throughput as $1/\text{hops}$, could not have seen this.

B. Deadlock

The CDG of BFS routing is cyclic on both the torus and the modified torus. On the mesh it happens to be acyclic, because breaking ties toward the lower neighbour index acts like a turn restriction. The up*/down* escape network is acyclic on all three.

Simulation agrees with the CDG analysis (Table II). With a single VC, BFS routing ran cleanly up to 15% load and then deadlocked in every run at 20% and

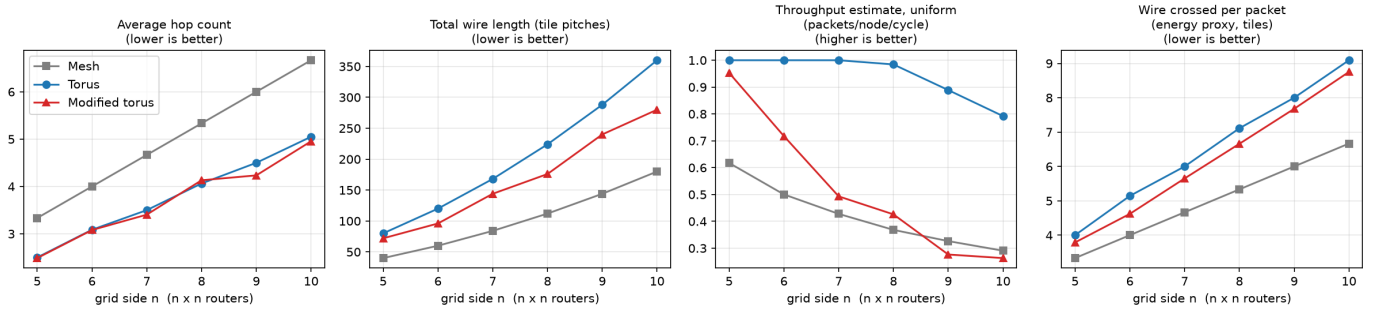


Fig. 2. Analytical metrics against grid size. The modified torus matches the torus on hop count and saves 10–22% of its wire, but its throughput estimate drops toward, and for $n \geq 9$ below, that of the mesh.

TABLE II
Fraction of runs that deadlocked (8×8 , uniform traffic, five seeds per point).

Routing / topology	Injection rate (packets/node/cycle)					
	0.05	0.10	0.15	0.20	0.30	0.50
BFS, 1 VC / mesh	0	0	0	0	0	0
BFS, 1 VC / torus	0	0	0	1	1	1
BFS, 1 VC / modified	0	0	0	1	1	1
Adaptive + escape / all	0	0	0	0	0	0

TABLE III
Saturation throughput (packets/node/cycle) and uniform zero-load latency T_0 (cycles), 8×8 , one-cycle links. “Searched” is the 208-tile design from Section V-E.

Topology	Unif.	Transp.	Bit-c.	Torn.	T_0
Mesh	0.40	0.33	0.17	0.28	10.6
Torus	0.72	0.52	0.32	0.25	8.1
Modified torus	0.40	0.30	0.10	0.37	8.3
Searched (208)	0.45	0.28	0.20	0.28	7.5

above, typically within a few hundred cycles. The adaptive scheme never deadlocked, including at injection rates of 0.6 and 1.0, far past saturation. Every simulation result below uses it.

C. Simulator validation

A custom simulator needs an outside check. BookSim [13] cannot run our escape-channel routing, so the comparison uses routing both simulators support. We added dimension-order routing to our simulator, following BookSim’s `dim_order_mesh` and `dim_order_torus`, including its rule for splitting VCs between the two sides of the torus dateline. We matched the remaining settings: 4-packet buffers, single-flit packets, Bernoulli injection, 1-cycle links, and a router delay chosen so that both simulators charge 4 cycles per hop. BookSim also adds 6 cycles per packet for injection and ejection, and lets nodes send packets to themselves ($1/64$ of uniform traffic and $1/8$ of transpose). We corrected for both explicitly rather than tuning parameters until the curves met.

Fig. 3 shows the result for three traffic patterns. On the mesh with 2 VCs and the torus with 4 VCs, zero-load latency agrees to within 2%, the mean latency error below saturation is 0.2–3.2%, and saturation points differ by at most 0.06 packets/node/cycle (0.56 against 0.62 on the uniform torus, and within 0.02 elsewhere). On the torus with 2 VCs, where each dateline class has a single VC, our simulator saturates much later: 0.42 against 0.30 under uniform traffic. A mesh with one VC shows a similar gap: BookSim is unstable at 0.20, while our simulator stays stable up to 0.25. The cause is that in BookSim each

packet must win VC allocation and then switch allocation, so a single VC cannot forward packets back to back every cycle; our simulator allows it to. Our numbers are therefore optimistic whenever a traffic class has only one VC. The main experiments use three VCs, one for escape and two adaptive, which is the regime where the two simulators agree.

D. Cycle-level performance

Table III and Fig. 4 show the simulated behaviour. The modified torus behaves like a torus at low load and like a mesh near saturation. Its zero-load latency under uniform traffic is 8.3 cycles, against 8.1 for the torus and 10.6 for the mesh. Its saturation throughput is 0.40, the same as the mesh and 45% below the torus. The ordering under transpose is the same.

The traffic pattern matters a great deal. Under tornado traffic, where every node sends to the node $\lceil n/2 \rceil - 1$ columns to its right (wrapping around), the modified torus saturates at 0.37 against 0.25 for the torus. We suspect this is because the folded links on the top and bottom rows span three columns, exactly the distance tornado traffic travels at $n = 8$, but we have not analysed it further. Bit-complement is the opposite case. Every packet has to cross the centre of the chip, the folded links run along the edges and do little for that traffic, and the modified torus saturates at 0.10, below even the mesh.

Charging long links their real delay (bottom row of Fig. 4) moves the zero-load results in the expected direction. The modified torus becomes slightly faster than the torus at low load: 9.4 against 9.6 cycles for uniform

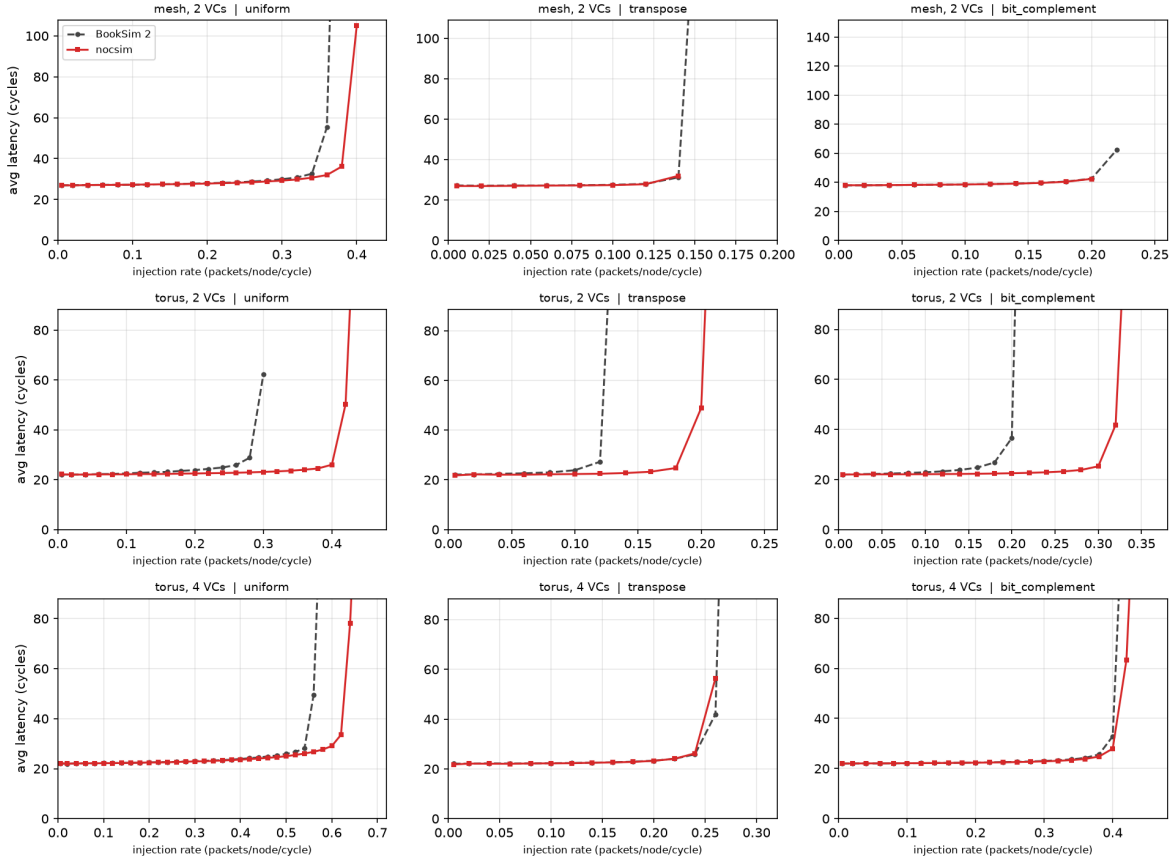


Fig. 3. Our simulator (red) against BookSim 2 (grey, dashed) with dimension-order routing on 8×8 networks, averaged over three seeds. The curves agree whenever each traffic class has at least two VCs (top and bottom rows). With one VC per dateline class (middle row) our simulator saturates later.

TABLE IV

Search at the modified torus' wire budget (176 tiles, 8×8), 2500 evaluations, three seeds. Throughput is the mean of the four per-pattern estimates, relative to the torus.

Method	J (mean $\pm$ std)	Thr./torus	Hops
Random search	-0.03 ± 0.08	0.42	3.92
REINFORCE	0.25 ± 0.03	0.50	4.10
Simulated annealing	0.34 ± 0.02	0.59	4.04
Modified torus (hand)	0.37	0.63	4.13

traffic and 10.2 against 10.7 for transpose. These gains are small, and saturation throughput is almost unchanged.

The simulator generally achieves 70–95% of the analytical estimate $\hat{\Theta}$. The torus, at 74% under uniform traffic, is at the low end. Adaptive routing occasionally exceeds the estimate (mesh and modified torus under tornado). This is why we treat $\hat{\Theta}$ only as a search heuristic and check its conclusions in simulation.

E. Searching for better layouts

At the modified torus's own budget of 176 tiles, no method found a better layout (Table IV, Fig. 5). Annealing from random starts came closest, at $J = 0.34$

against 0.37. Annealing started from the modified torus never found anything better than its starting point. REINFORCE came out between random search and annealing. We think the main reason is that its policy is tabular. It learns a separate logit for every pair of ports and has no way to see that two pairings are geometrically similar, so 2500 episodes are not enough. A policy that shares parameters across positions, such as a graph network over the floorplan, would be the obvious next step.

Fig. 6 sweeps the budget. The best designs found trace a frontier that the modified torus sits on, and it is steep near the torus. Going from 208 to 224 tiles (8% more wire) raises the throughput estimate from 0.71 to 1.0 of the torus's. At 224 tiles the best design found is the torus itself, reached by annealing started from it; cold starts did not get there. Below 176 tiles the designs found have fewer hops than a mesh but lower throughput. This is a limit of the design space, which requires every spare port to be used, so leaving ports idle would sometimes be the better choice.

The 208-tile design (Fig. 7) is a useful intermediate point. It is irregular and uses 7% less wire than the torus. It has the lowest zero-load latency of the four

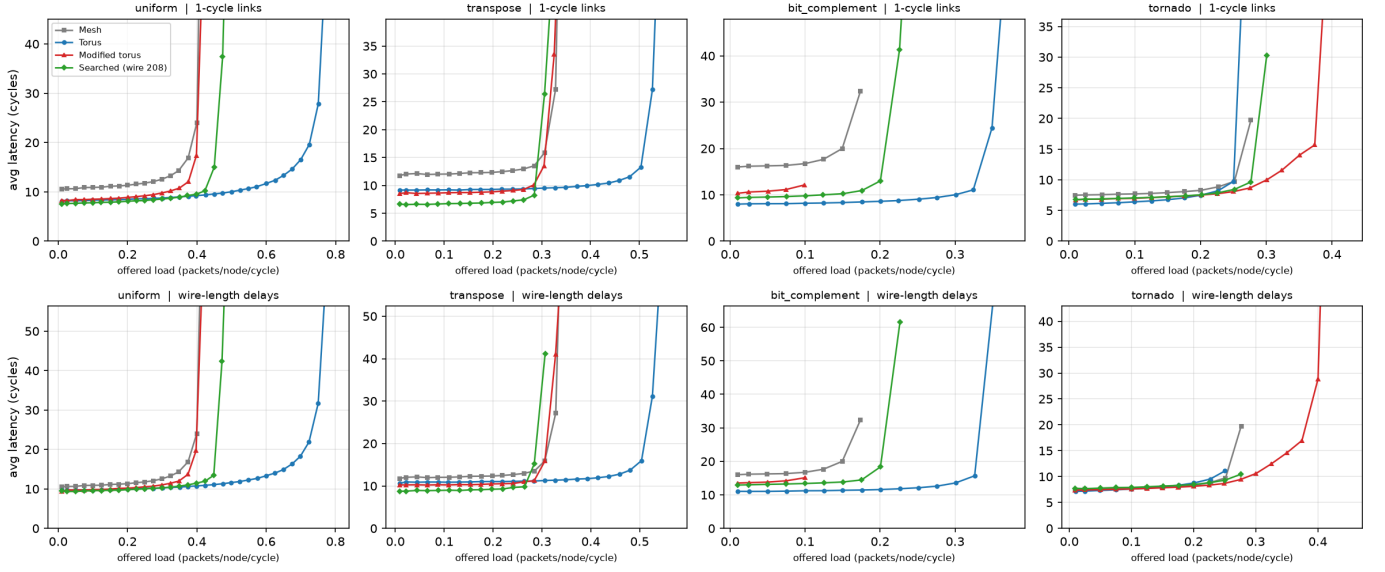


Fig. 4. Average latency against offered load for an 8×8 network with adaptive routing. Top row: every link takes one cycle. Bottom row: a link of length L takes $\lceil L/2 \rceil$ cycles.

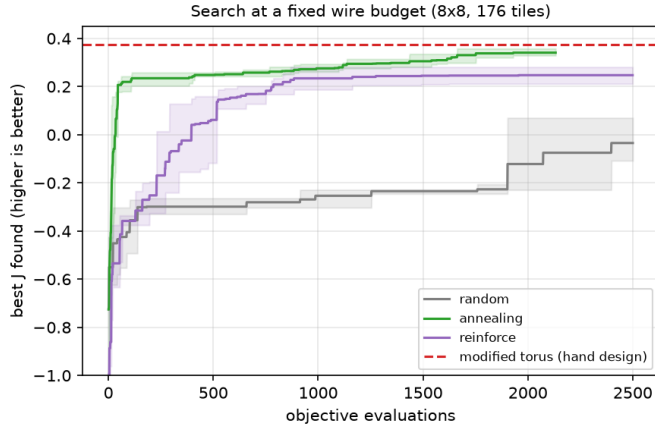


Fig. 5. Best objective found against number of evaluations at a 176-tile budget. Shaded bands show the range over three seeds.

networks (7.5 cycles uniform, 6.7 transpose) and twice the bit-complement throughput of the modified torus. It is still well short of the torus on saturation throughput (Table III).

One lesson from the search concerns the objective. Our first version optimised the uniform-traffic estimate alone. At 176 tiles it returned a layout with a 13% higher estimate than the modified torus, which looked like a clear improvement. In a preliminary simulation, that layout was indeed better on bit-complement (0.18 against 0.10), but it was 16% worse on transpose. Scoring designs on a mix of traffic patterns removed the problem, and with it the apparent improvement.

VI. Limitations

Our wire model is the Manhattan length on a flat floorplan. A folded layout would change the comparison with the torus, and we have no area or power model beyond counting wire tiles. The simulator uses single-flit packets and a simple greedy allocator, so absolute numbers should not be compared with wormhole-switched results from BookSim or Garnet. Its one known gap with BookSim, single-VC throughput (Section V-C), makes it optimistic for configurations with one VC per traffic class. All traffic is synthetic. Whether the modified torus is worth building depends on whether real workloads look more like tornado or like bit-complement, and that needs application traces. Finally, the search covers only boundary-link layouts at radix 4 on an 8×8 grid.

VII. Conclusion

The modified torus does what it was designed to do. It removes a fifth of the torus’s wire without increasing the hop count. Its original evaluation missed the cost: about half of the torus’s saturation throughput, and a routing algorithm that deadlocks. The design has a clear niche, tornado-like traffic and wire-constrained floorplans, and at its own wire budget neither annealing nor reinforcement learning found a better layout. The broader lesson is a methodological one. Hop counts alone, or any single cheap objective, can make a design look better than it is, and the conclusions changed each time we added a more realistic measurement.

Acknowledgment

The modified torus topology was first proposed in a B.Tech. thesis at Assam University with S. Sharma and

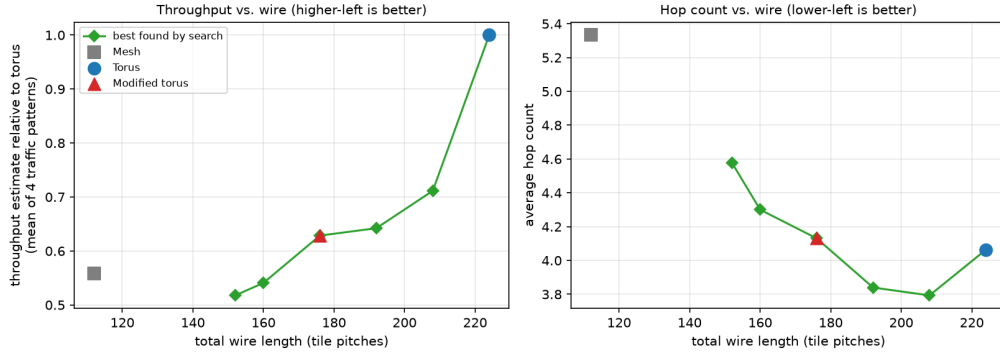


Fig. 6. Best design found for each wire budget (green), compared with the three reference topologies. Throughput is the mean of the four per-pattern estimates relative to the torus.

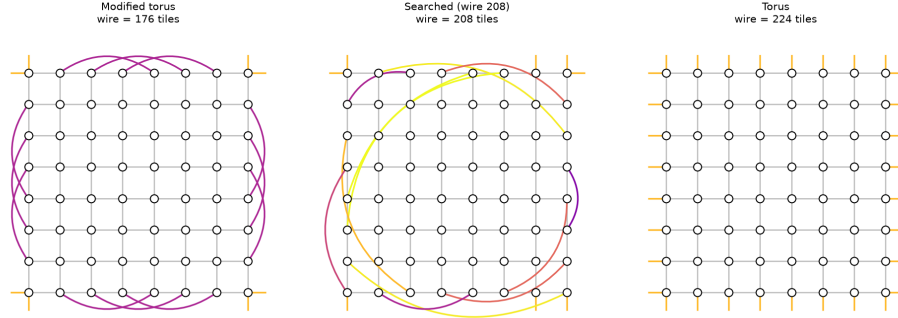


Fig. 7. The modified torus (left), the best 208-tile design found by annealing (centre), and the torus (right).

B. R. S. Satyanarayana, supervised by Dr. A. Biswas [1]. The generalisation, simulator, deadlock analysis, search and all experiments in this paper are new.

References

- [1] S. Sharma, R. Ray, and B. R. S. Satyanarayana, "Study on BFS based routing of a new modified torus NoC topology," B.Tech. thesis, Dept. of Computer Science and Engineering, Assam University, Silchar, India, 2023.
- [2] W. J. Dally and B. Towles, "Route packets, not wires: On-chip interconnection networks," in Proc. Design Automation Conf. (DAC), 2001, pp. 684–689.
- [3] S. Kumar, A. Jantsch, J.-P. Soininen, M. Forsell, M. Millberg, J. Öberg, K. Tiensyrjä, and A. Hemani, "A network on chip architecture and design methodology," in Proc. IEEE Computer Society Annual Symp. on VLSI, 2002, pp. 105–112.
- [4] W. J. Dally and B. Towles, Principles and Practices of Interconnection Networks. San Francisco, CA, USA: Morgan Kaufmann, 2004.
- [5] A. Kumar, L.-S. Peh, P. Kundu, and N. K. Jha, "Express virtual channels: Towards the ideal interconnection fabric," in Proc. Int. Symp. on Computer Architecture (ISCA), 2007.
- [6] J. Kim, J. Balfour, and W. J. Dally, "Flattened butterfly topology for on-chip networks," in Proc. IEEE/ACM Int. Symp. on Microarchitecture (MICRO), 2007.
- [7] W. J. Dally and C. L. Seitz, "Deadlock-free message routing in multiprocessor interconnection networks," IEEE Trans. Computers, vol. C-36, no. 5, pp. 547–553, 1987.
- [8] J. Duato, "A new theory of deadlock-free adaptive routing in wormhole networks," IEEE Trans. Parallel and Distributed Systems, vol. 4, no. 12, pp. 1320–1331, 1993.
- [9] M. D. Schroeder et al., "Autonet: A high-speed, self-configuring local area network using point-to-point links," IEEE J. Selected Areas in Communications, vol. 9, no. 8, pp. 1318–1335, 1991.
- [10] S. Kirkpatrick, C. D. Gelatt, and M. P. Vecchi, "Optimization by simulated annealing," Science, vol. 220, no. 4598, pp. 671–680, 1983.
- [11] R. J. Williams, "Simple statistical gradient-following algorithms for connectionist reinforcement learning," Machine Learning, vol. 8, pp. 229–256, 1992.
- [12] J. A. Boyan and M. L. Littman, "Packet routing in dynamically changing networks: A reinforcement learning approach," in Advances in Neural Information Processing Systems 6, 1994, pp. 671–678.
- [13] N. Jiang et al., "A detailed and flexible cycle-accurate network-on-chip simulator," in Proc. IEEE Int. Symp. on Performance Analysis of Systems and Software (ISPASS), 2013.
- [14] N. Agarwal, T. Krishna, L.-S. Peh, and N. K. Jha, "GARNET: A detailed on-chip network model inside a full-system simulator," in Proc. IEEE Int. Symp. on Performance Analysis of Systems and Software (ISPASS), 2009.